

第七章 常见设计模式（面试突击★★★★）

设计模式是软件工程中反复出现的通用解决方案。本章聚焦高频面试设计模式，每个模式都配有完整可编译的C++17代码。

目录

- 1. [单例模式 \(Singleton\)](#)
- 2. [工厂模式 \(Factory Method\)](#)
- 3. [抽象工厂 \(Abstract Factory\)](#)
- 4. [观察者模式 \(Observer\)](#)
- 5. [策略模式 \(Strategy\)](#)
- 6. [责任链模式 \(Chain of Responsibility\)](#)
- 7. [模板方法模式 \(Template Method\)](#)
- 8. [其他模式简述](#)
- 9. [设计模式对比总结](#)
- 10. [面试核心](#)

1. 单例模式 (Singleton)

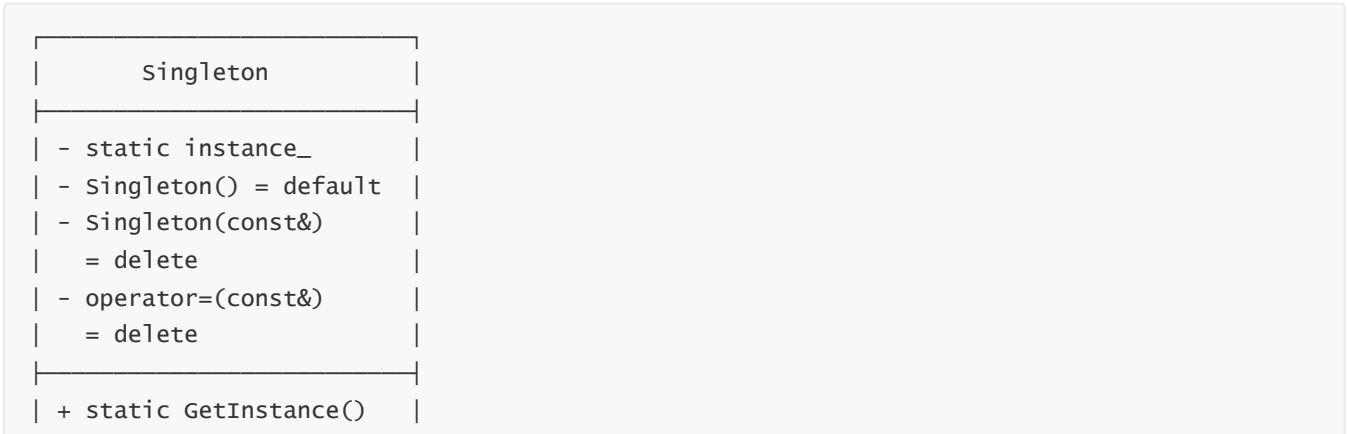
1.1 思想

确保一个类只有一个实例，并提供一个全局访问点。

核心要素：

- 私有构造函数（防止外部创建）
- 删除拷贝构造和拷贝赋值（防止拷贝出新实例）
- 静态成员函数返回唯一实例的引用
- 线程安全（C++11及以后）

1.2 类图



```
| -> Singleton&  
| + DoSomething()  
|
```

1.3 代码实现

A. Meyers Singleton (C++11静态局部变量, 推荐)

```
#include <iostream>  
#include <mutex>  
#include <thread>  
#include <vector>  
  
// =====  
// Meyers Singleton - C++11 保证 static 局部变量初始化线程安全  
// =====  
class MeyersSingleton {  
public:  
    // C++11: static 局部变量初始化由编译器保证线程安全  
    static MeyersSingleton& GetInstance() {  
        static MeyersSingleton instance;  
        return instance;  
    }  
  
    void DoWork() {  
        std::cout << "MeyersSingleton working, addr=" << this << std::endl;  
    }  
  
    int GetValue() const { return value_; }  
    void SetValue(int v) { value_ = v; }  
  
    // 禁止拷贝和移动  
    MeyersSingleton(const MeyersSingleton&) = delete;  
    MeyersSingleton& operator=(const MeyersSingleton&) = delete;  
    MeyersSingleton(MeyersSingleton&&) = delete;  
    MeyersSingleton& operator=(MeyersSingleton&&) = delete;  
  
private:  
    MeyersSingleton() {  
        std::cout << "MeyersSingleton constructed" << std::endl;  
    }  
    ~MeyersSingleton() {  
        std::cout << "MeyersSingleton destroyed" << std::endl;  
    }  
  
    int value_ = 0;  
};  
  
// 使用示例  
void TestMeyersSingleton() {  
    MeyersSingleton& s1 = MeyersSingleton::GetInstance();  
    MeyersSingleton& s2 = MeyersSingleton::GetInstance();
```

```

s1.SetValue(42);
std::cout << "s2 value = " << s2.GetValue() << std::endl; // 42
std::cout << "Same instance? " << (&s1 == &s2 ? "yes" : "no") << std::endl;
}

```

B. 双重检查锁定 (Double-Checked Locking)

```

#include <atomic>
#include <mutex>

// =====
// 双重检查锁定 - C++11 之前的老式写法
// C++11 之后推荐 Meyers Singleton, 但面试常考此写法
// =====
class DCLSingleton {
public:
    static DCLSingleton* GetInstance() {
        // 第一次检查: 避免每次都加锁
        DCLSingleton* tmp = instance_.load(std::memory_order_acquire);
        if (tmp == nullptr) {
            std::lock_guard<std::mutex> lock(mutex_);
            // 第二次检查: 防止多线程同时通过第一次检查
            tmp = instance_.load(std::memory_order_relaxed);
            if (tmp == nullptr) {
                tmp = new DCLSingleton();
                instance_.store(tmp, std::memory_order_release);
            }
        }
        return tmp;
    }

    void DoWork() {
        std::cout << "DCLSingleton working" << std::endl;
    }

    DCLSingleton(const DCLSingleton&) = delete;
    DCLSingleton& operator=(const DCLSingleton&) = delete;

private:
    DCLSingleton() = default;
    ~DCLSingleton() = default;

    static std::atomic<DCLSingleton*> instance_;
    static std::mutex mutex_;
};

std::atomic<DCLSingleton*> DCLSingleton::instance_{nullptr};
std::mutex DCLSingleton::mutex_;

// 注意: DCLSingleton 不会自动析构, 需要注意资源泄漏问题

```

C. 带销毁的懒汉单例（使用 atexit 或嵌套静态类）

```
#include <cstdlib>

class SafeSingleton {
public:
    static SafeSingleton& GetInstance() {
        static SafeSingleton instance;
        return instance;
    }

    // 手动销毁（如果顺序有要求）
    static void Destroy() {
        // Meyers Singleton 会在程序结束时自动析构
        // 这里仅演示显式控制的思路
    }

    SafeSingleton(const SafeSingleton&) = delete;
    SafeSingleton& operator=(const SafeSingleton&) = delete;

private:
    SafeSingleton() {
        // 注册 atexit 保证析构
        std::atexit([]() {
            std::cout << "SafeSingleton atexit cleanup" << std::endl;
        });
    }
    ~SafeSingleton() = default;
};
```

1.4 使用场景

场景	说明
日志系统	整个程序共用一个日志实例
配置管理	全局配置，一处加载多处读取
连接池	数据库连接池全局唯一
线程池	全局线程池避免重复创建
设备管理器	如显卡管理器、打印机管理器
游戏引擎	GameManager 等全局管理类

1.5 优缺点

优点：

- 严格控制唯一实例，节省资源
- 提供全局访问点，方便调用

- 延迟初始化（懒汉式），按需创建

缺点：

- 全局状态 → 单元测试困难（测试间互相影响）
- 隐式依赖 → 降低代码可读性和可维护性
- 生命周期管理复杂 → 多个单例的析构顺序问题
- 难以扩展 → 假设"只有一个"可能在未来被推翻
- 多线程环境需要额外处理

1.6 面试题

Q1: 为什么删除拷贝构造函数和拷贝赋值运算符？

如果不删除拷贝构造和拷贝赋值，用户可能写出：

```
MeyersSingleton copy = MeyersSingleton::GetInstance(); // 拷贝出一个新实例
```

这违反了单例的唯一性原则。C++11 最佳实践是用 `= delete` 显式删除，同时也删除移动构造和移动赋值。

Q2: Meyers Singleton 为什么是线程安全的？

C++11 标准 ([stmt.dcl]/4) 规定：如果多个线程同时进入同一个 `static` 局部变量的初始化，该初始化只会执行一次，其他线程会阻塞等待初始化完成。编译器使用类似双重检查锁定的机制实现，但无锁开销。这就是 Meyers Singleton (Scott Meyers 提出) 成为 C++11 后最佳单例写法的原因。

Q3: 单例模式有什么替代方案？

1. **依赖注入**：将"单例"对象通过构造函数参数传递，由 DI 容器管理生命周期
2. **纯函数 + 无状态**：如果单例只是工具函数集合，直接用命名空间 + 自由函数
3. **Monostate 模式**：所有成员都是 `static`，但可以正常构造（不常用）
4. **局部上下文对象**：不暴露全局，通过参数显式传递

2. 工厂模式 (Factory Method)

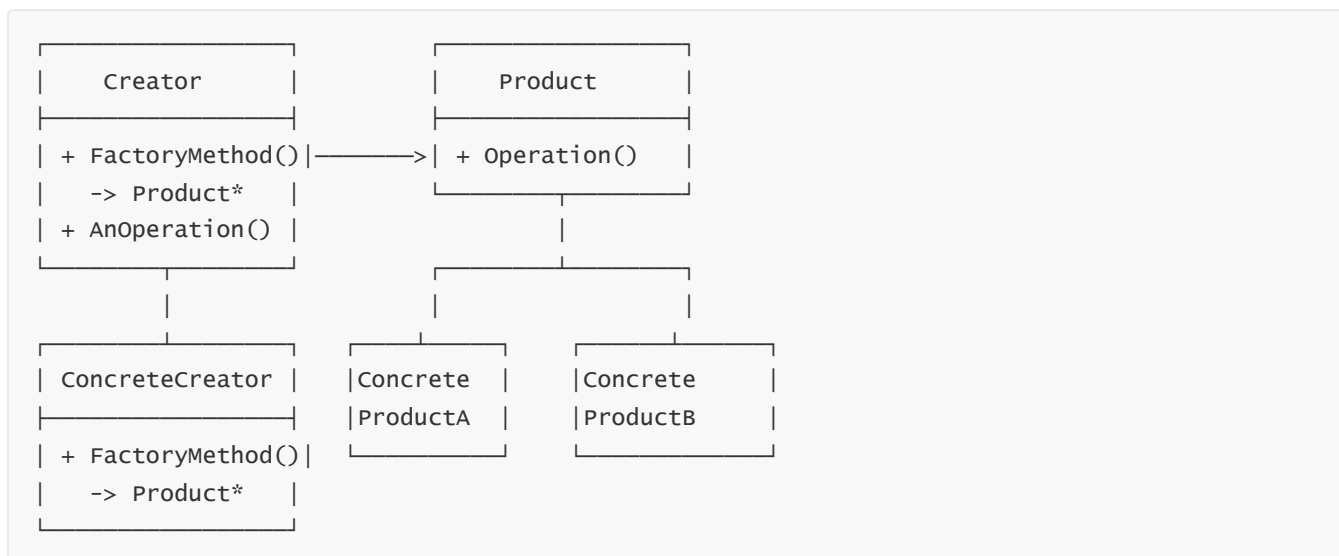
2.1 思想

定义一个创建对象的接口，让子类决定实例化哪个类。工厂方法使得类的实例化延迟到子类。

与直接使用构造函数的区别：

- 构造函数必须返回具体类型，工厂可以返回基类指针/引用（多态）
- 构造函数必须有具体类名，工厂可以通过字符串/枚举/配置决定创建哪个类
- 工厂可以包含复杂的创建逻辑（参数校验、缓存、池化等）

2.2 类图



2.3 代码实现

A. 简单工厂 (Simple Factory) — 不是 GoF 模式，但常用

```
#include <iostream>
#include <memory>
#include <string>

// =====
// 简单工厂 - 一个工厂类根据参数创建不同产品
// =====

// 产品基类
class Transport {
public:
    virtual ~Transport() = default;
    virtual void Deliver() const = 0;
};

// 具体产品
class Truck : public Transport {
public:
    void Deliver() const override {
        std::cout << "Truck delivering by road" << std::endl;
    }
};

class Ship : public Transport {
public:
    void Deliver() const override {
        std::cout << "Ship delivering by sea" << std::endl;
    }
};

class Plane : public Transport {
```

```

public:
    void Deliver() const override {
        std::cout << "Plane delivering by air" << std::endl;
    }
};

// 简单工厂
enum class TransportType { Truck, Ship, Plane };

class SimpleTransportFactory {
public:
    static std::unique_ptr<Transport> CreateTransport(TransportType type) {
        switch (type) {
            case TransportType::Truck:
                return std::make_unique<Truck>();
            case TransportType::Ship:
                return std::make_unique<Ship>();
            case TransportType::Plane:
                return std::make_unique<Plane>();
            default:
                throw std::invalid_argument("Unknown transport type");
        }
    }
};

void TestSimpleFactory() {
    auto truck = SimpleTransportFactory::CreateTransport(TransportType::Truck);
    truck->Deliver();

    auto ship = SimpleTransportFactory::CreateTransport(TransportType::Ship);
    ship->Deliver();
}

```

B. 工厂方法模式 (Factory Method) — GoF 模式

```

#include <iostream>
#include <memory>

// =====
// 工厂方法模式 - 每个子类重写工厂方法
// =====

// 产品基类
class Logger {
public:
    virtual ~Logger() = default;
    virtual void Log(const std::string& msg) = 0;
};

// 具体产品
class FileLogger : public Logger {
public:

```

```

    void Log(const std::string& msg) override {
        std::cout << "[FILE] " << msg << std::endl;
    }
};

class ConsoleLogger : public Logger {
public:
    void Log(const std::string& msg) override {
        std::cout << "[CONSOLE] " << msg << std::endl;
    }
};

class NetworkLogger : public Logger {
public:
    void Log(const std::string& msg) override {
        std::cout << "[NETWORK] " << msg << std::endl;
    }
};

// 创建者基类 - 工厂方法模式的核心
class LoggerFactory {
public:
    virtual ~LoggerFactory() = default;

    // 工厂方法 - 子类实现
    virtual std::unique_ptr<Logger> CreateLogger() = 0;

    // 模板方法: 展示工厂方法的典型用法
    void DoSomeLogging(const std::string& msg) {
        auto logger = CreateLogger(); // 工厂方法
        logger->Log(msg);
        // 可能还有统一的后处理逻辑
    }
};

// 具体创建者
class FileLoggerFactory : public LoggerFactory {
public:
    std::unique_ptr<Logger> CreateLogger() override {
        return std::make_unique<FileLogger>();
    }
};

class ConsoleLoggerFactory : public LoggerFactory {
public:
    std::unique_ptr<Logger> CreateLogger() override {
        return std::make_unique<ConsoleLogger>();
    }
};

void TestFactoryMethod() {
    ConsoleLoggerFactory consoleFactory;
    consoleFactory.DoSomeLogging("Hello from console");
}

```



```
FileLoggerFactory fileFactory;
fileFactory.DoSomeLogging("Hello from file");
}
```

2.4 使用场景

- 日志框架：根据配置创建不同的 Logger
- 数据库访问层：根据数据库类型（MySQL/PostgreSQL/SQLite）创建不同的 Connection
- GUI 框架：不同操作系统创建不同的 Button/Window
- 序列化框架：根据格式（JSON/XML/Protobuf）创建不同的 Serializer

2.5 优缺点

优点：

- 解耦：创建者与具体产品解耦，符合依赖倒置原则
- 扩展性：新增产品只需新增子类，符合开闭原则
- 单一职责：创建逻辑集中，方便维护

缺点：

- 类爆炸：每个新产品需要一个新工厂类，类数量翻倍
- 复杂度增加：简单场景直接用 `std::make_unique` 更直观
- 不直观：代码阅读者需要跟踪工厂的继承链

2.6 面试题

Q1: 为什么不用构造函数，要用工厂？

1. **多态返回**：构造函数只能返回自身类型，工厂可以返回基类指针
2. **运行时选择**：构造函数在编译时确定，工厂可以运行时按配置决定创建哪个子类
3. **对象池/缓存**：工厂可以复用已有对象，构造函数每次 new
4. **复杂校验**：工厂可以在创建前后做参数校验、日志、性能统计
5. **命名语义**：`CreateTruck()` 比 `new Truck()` 更具表达力

Q2: 简单工厂和工厂方法有什么区别？

维度	简单工厂	工厂方法
GoF 定义	不是 GoF 模式	是 GoF 创建型模式
扩展方式	修改工厂类的 switch	新增工厂子类
开闭原则	违反（需修改工厂类）	遵循（新增类即可）
复杂度	低	高
适用场景	产品类型少且稳定	产品族多且经常扩展

Q3: 工厂模式在 C++ 中可以用模板替代吗？

可以。模板工厂在编译期确定类型，性能更好，但没有运行时多态能力：

```
template <typename T>
std::unique_ptr<T> Create() {
    return std::make_unique<T>();
}
```

选择原则：编译期能确定的用模板，需要运行时配置的用虚函数工厂。

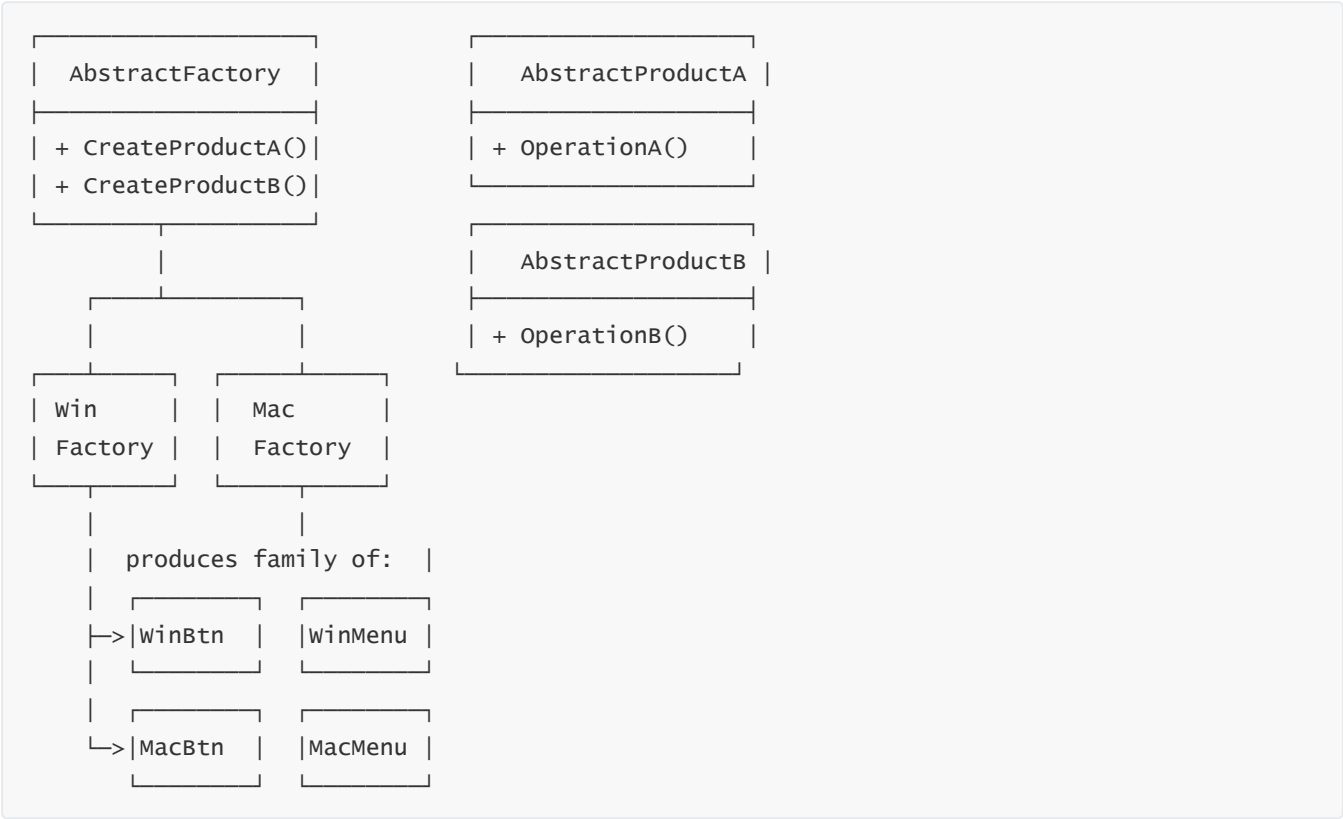
3. 抽象工厂 (Abstract Factory)

3.1 思想

提供一个接口，创建一系列相关或相互依赖的对象，而无需指定具体类。

工厂方法创建一个产品，抽象工厂创建一组产品族。

3.2 类图



3.3 代码实现

```
#include <iostream>
#include <memory>
#include <string>

// =====
```

```

// 抽象工厂 - 创建整个 GUI 组件族
// =====

// ----- 抽象产品 A: 按钮 -----
class Button {
public:
    virtual ~Button() = default;
    virtual void Paint() const = 0;
};

// ----- 抽象产品 B: 复选框 -----
class CheckBox {
public:
    virtual ~CheckBox() = default;
    virtual void Paint() const = 0;
};

// ----- 抽象产品 C: 文本框 -----
class TextEdit {
public:
    virtual ~TextEdit() = default;
    virtual void Paint() const = 0;
};

// ----- windows 具体产品 -----
class WinButton : public Button {
public:
    void Paint() const override {
        std::cout << " [Windows Button] styled with Fluent Design" << std::endl;
    }
};

class WinCheckBox : public CheckBox {
public:
    void Paint() const override {
        std::cout << " [Windows CheckBox] rounded corners" << std::endl;
    }
};

class WinTextEdit : public TextEdit {
public:
    void Paint() const override {
        std::cout << " [Windows TextEdit] with spell check" << std::endl;
    }
};

// ----- macOS 具体产品 -----
class MacButton : public Button {
public:
    void Paint() const override {
        std::cout << " [Mac Button] styled with Aqua" << std::endl;
    }
};

```

```

class MacCheckBox : public CheckBox {
public:
    void Paint() const override {
        std::cout << " [Mac CheckBox] circle check" << std::endl;
    }
};

class MacTextEdit : public TextEdit {
public:
    void Paint() const override {
        std::cout << " [Mac TextEdit] with auto-correction" << std::endl;
    }
};

// ----- 抽象工厂 -----
class GUIFactory {
public:
    virtual ~GUIFactory() = default;

    virtual std::unique_ptr<Button> CreateButton() = 0;
    virtual std::unique_ptr<CheckBox> CreateCheckBox() = 0;
    virtual std::unique_ptr<TextEdit> CreateTextEdit() = 0;
};

// ----- 具体工厂: windows -----
class WinFactory : public GUIFactory {
public:
    std::unique_ptr<Button> CreateButton() override {
        return std::make_unique<WinButton>();
    }
    std::unique_ptr<CheckBox> CreateCheckBox() override {
        return std::make_unique<WinCheckBox>();
    }
    std::unique_ptr<TextEdit> CreateTextEdit() override {
        return std::make_unique<WinTextEdit>();
    }
};

// ----- 具体工厂: Mac -----
class MacFactory : public GUIFactory {
public:
    std::unique_ptr<Button> CreateButton() override {
        return std::make_unique<MacButton>();
    }
    std::unique_ptr<CheckBox> CreateCheckBox() override {
        return std::make_unique<MacCheckBox>();
    }
    std::unique_ptr<TextEdit> CreateTextEdit() override {
        return std::make_unique<MacTextEdit>();
    }
};

```

```
// ----- 客户端代码 -----
class Application {
public:
    explicit Application(GUIFactory& factory) : factory_(factory) {}

    void RenderUI() {
        auto button = factory_.CreateButton();
        auto checkbox = factory_.CreateCheckBox();
        auto textedit = factory_.CreateTextEdit();

        std::cout << "Rendering UI with " << typeid(factory_).name() << ":" << std::endl;
        button->Paint();
        checkbox->Paint();
        textedit->Paint();
    }

private:
    GUIFactory& factory_;
};

void TestAbstractFactory() {
    std::cout << "=== Windows Mode ===" << std::endl;
    WinFactory winFactory;
    Application winApp(winFactory);
    winApp.RenderUI();

    std::cout << std::endl << "=== Mac Mode ===" << std::endl;
    MacFactory macFactory;
    Application macApp(macFactory);
    macApp.RenderUI();
}
```

3.4 使用场景

- 跨平台 UI 框架 (Qt、wxWidgets 的底层思路)
- 不同数据库后端的 ORM (MySQL、PostgreSQL、SQLite 的 DAO 工厂)
- 主题系统：换肤时整体切换一组控件
- 游戏引擎：不同渲染后端 (OpenGL、Vulkan、DirectX)
- 不同格式的文档生成器 (PDF、HTML、Markdown)

3.5 优缺点

优点：

- 保证产品族的一致性 (不会出现 Windows 按钮 + Mac 复选框的组合)
- 客户端与具体产品解耦，切换产品族只需要换一个工厂
- 符合开闭原则：新增产品族 (如 LinuxFactory) 不需要改客户端代码

缺点：

- 新增产品 (如加一个 Slider) 需要改抽象工厂接口和所有具体工厂

- 类数量随产品族和产品数量乘积增长
- 抽象层级较深，调试困难

3.6 面试题

Q1: 抽象工厂和工厂方法的区别？

维度	工厂方法	抽象工厂
创建数量	1种产品	多种产品（产品族）
维度	单维度扩展（加产品）	双维度（加产品族 vs 加产品）
抽象层级	1个抽象方法	多个抽象方法
关系	是抽象工厂的子集	包含多个工厂方法

Q2: 抽象工厂如何支持"增加新产品"？

默认不支持——抽象工厂接口是固定的。如果确实需要，有两种方案：

1. 使用可扩展的抽象工厂（每个产品有独立的工厂接口，而非一个大接口）
2. 结合原型模式（Prototype）：工厂不硬编码 CreateXxx 方法，而是从原型克隆

Q3: 什么情况下不应该使用抽象工厂？

- 产品族只有一种产品 → 用工厂方法
- 产品类型基本不变，但需要按参数选择 → 用简单工厂
- 不需要运行时多态 → 用模板

4. 观察者模式 (Observer)

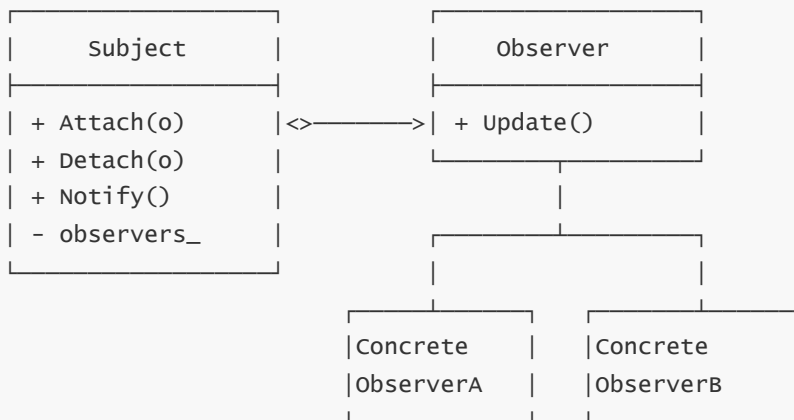
4.1 思想

定义对象间一对多的依赖关系，当一个对象状态变化时，所有依赖它的对象都得到通知并自动更新。

核心角色：

- **Subject (主题)**：维护观察者列表，状态变化时通知
- **Observer (观察者)**：定义更新接口，接收通知

4.2 类图



4.3 代码实现

```
#include <iostream>
#include <vector>
#include <memory>
#include <algorithm>
#include <string>
#include <functional>

// =====
// 观察者模式 - 完整的事件通知系统
// =====

// 前向声明
class Subject;

// ----- 观察者接口 -----
class IObserver {
public:
    virtual ~IObserver() = default;
    // Push 模型: 主题把数据推给观察者
    virtual void Update(const std::string& event, const std::string& data) = 0;
};

// ----- 主题 -----
class Subject {
public:
    virtual ~Subject() = default;

    void Attach(std::shared_ptr<IObserver> observer) {
        if (std::find(observers_.begin(), observers_.end(), observer)
            == observers_.end()) {
            observers_.push_back(observer);
        }
    }
}
```

```

void Detach(std::shared_ptr<IObserver> observer) {
    observers_.erase(
        std::remove(observers_.begin(), observers_.end(), observer),
        observers_.end()
    );
}

void Notify(const std::string& event, const std::string& data) {
    // 遍历副本防止回调中修改 observers_ 导致迭代器失效
    auto observersCopy = observers_;
    for (auto& obs : observersCopy) {
        if (obs) {
            obs->Update(event, data);
        }
    }
}

private:
    std::vector<std::shared_ptr<IObserver>> observers_;
};

// ----- 具体主题: 股票行情 -----
class StockMarket : public Subject {
public:
    void SetPrice(const std::string& symbol, double price) {
        prices_[symbol] = price;
        std::string data = symbol + ":" + std::to_string(price);
        Notify("PRICE_CHANGE", data);
    }

    double GetPrice(const std::string& symbol) const {
        auto it = prices_.find(symbol);
        return (it != prices_.end()) ? it->second : 0.0;
    }

private:
    std::unordered_map<std::string, double> prices_;
};

// ----- 具体观察者 -----
class Trader : public IObserver {
public:
    explicit Trader(const std::string& name) : name_(name) {}

    void Update(const std::string& event, const std::string& data) override {
        std::cout << "[Trader " << name_ << "] Received " << event
            << ": " << data << std::endl;
    }

private:
    std::string name_;
};

```



```

class AlertSystem : public IObserver {
public:
    explicit AlertSystem(double threshold) : threshold_(threshold) {}

    void Update(const std::string& event, const std::string& data) override {
        // data format: "SYMBOL:PRICE"
        auto pos = data.find(':');
        std::string symbol = data.substr(0, pos);
        double price = std::stod(data.substr(pos + 1));

        if (price > threshold_) {
            std::cout << "[ALERT] " << symbol << " price $" << price
                << " exceeds threshold $" << threshold_ << "!" << std::endl;
        }
    }

private:
    double threshold_;
};

void TestObserver() {
    StockMarket market;

    auto trader1 = std::make_shared<Trader>("Alice");
    auto trader2 = std::make_shared<Trader>("Bob");
    auto alert = std::make_shared<AlertSystem>(100.0);

    market.Attach(trader1);
    market.Attach(trader2);
    market.Attach(alert);

    std::cout << "=== Price update 1 ===" << std::endl;
    market.SetPrice("AAPL", 95.0);

    std::cout << "=== Price update 2 ===" << std::endl;
    market.SetPrice("AAPL", 105.0);

    std::cout << "=== Detach trader2 ===" << std::endl;
    market.Detach(trader2);

    std::cout << "=== Price update 3 ===" << std::endl;
    market.SetPrice("GOOGL", 120.0);
}

```

Pull 模型变体

```
// Pull 模型：观察者主动从主题拉取数据
class IPullObserver {
public:
    virtual ~IPullObserver() = default;
    virtual void Update(Subject* subject) = 0; // 只传主题指针
};

// 实现：在 update 中通过 subject->GetXxx() 拉取需要的数据
```

4.4 使用场景

- GUI 事件处理：按钮点击 → 所有注册的回调
- 股票行情推送：价格变化 → 通知所有订阅者
- 消息队列：生产者-消费者模式的通知机制
- MVC 架构：Model 变化 → View 更新
- 游戏事件系统：玩家死亡 → UI、音效、计分板各自响应
- React/Vue 的响应式数据绑定（思想上类似）

4.5 优缺点

优点：

- 松耦合：Subject 和 Observer 只通过接口交互
- 广播通信：一对多通知，添加新观察者无需改 Subject
- 运行时动态订阅/取消

缺点：

- 通知顺序不可控
- 性能：大量观察者时逐个通知开销大
- 悬空引用：Observer 销毁前未 Detach 会导致崩溃
- 重入问题：通知链中修改 Subject 可能引发死循环
- 调试困难：调用链深且隐式

4.6 面试题

Q1: Push 模型 vs Pull 模型？

模型	优点	缺点
Push	观察者不需要知道 Subject 接口，直接拿到数据	可能推送无用数据，接口膨胀
Pull	观察者按需拉取、接口简洁	观察者需要知道 Subject 接口，耦合增加

实际项目中常混合使用：Push 传事件类型 + Subject 指针，Observer 根据需要 Pull 具体数据。

Q2: 如何防止"悬空引用"问题?

1. Observer 析构时在析构函数中 `subject->Detach(this)`
2. Subject 存储 `std::weak_ptr<Iobserver>` , 通知时先 `lock()`
3. 使用 `std::shared_ptr` + `std::enable_shared_from_this`
4. 使用信号槽库 (如 Boost.Signals2) , 自带连接管理

Q3: 如何处理重入 (Reentrancy) 问题?

```
void Notify(const std::string& event, const std::string& data) {  
    ++notifyDepth_; // 重入检测  
    if (notifyDepth_ > 1) {  
        // 推迟到外层调用结束后统一通知  
        pendingEvents_.emplace_back(event, data);  
        --notifyDepth_;  
        return;  
    }  
    for (auto& obs : observers_) { obs->Update(event, data); }  
    --notifyDepth_;  
    // 处理推迟的事件  
    if (notifyDepth_ == 0) {  
        auto events = std::move(pendingEvents_);  
        for (auto& [ev, dat] : events) { Notify(ev, dat); }  
    }  
}
```

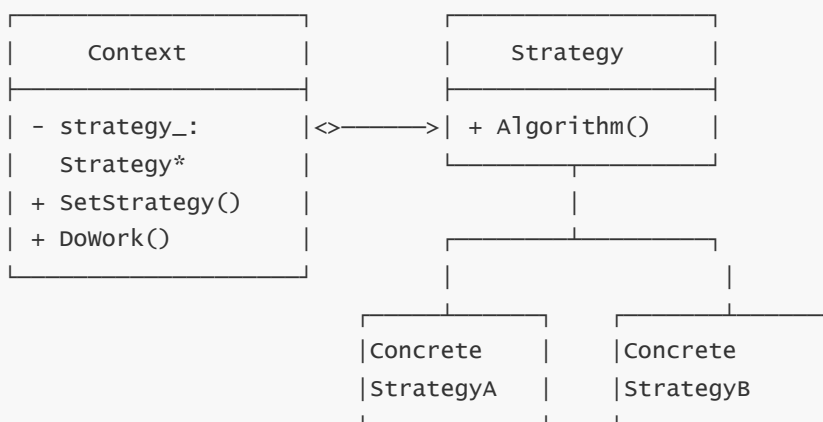
5. 策略模式 (Strategy)

5.1 思想

定义一系列算法，把它们封装起来，并让它们可以互相替换。策略模式让算法的变化独立于使用它的客户端。

核心思想：多态替换 — 把变化的部分抽象出来。

5.2 类图



5.3 代码实现

A. 动态策略 (运行时多态)

```
#include <iostream>
#include <memory>
#include <vector>
#include <algorithm>

// =====
// 动态策略 - 虚函数版本
// =====

// 排序策略接口
class SortStrategy {
public:
    virtual ~SortStrategy() = default;
    virtual void Sort(std::vector<int>& data) const = 0;
    virtual std::string Name() const = 0;
};

class BubbleSort : public SortStrategy {
public:
    void Sort(std::vector<int>& data) const override {
        int n = data.size();
        for (int i = 0; i < n - 1; ++i) {
            for (int j = 0; j < n - i - 1; ++j) {
                if (data[j] > data[j + 1]) {
                    std::swap(data[j], data[j + 1]);
                }
            }
        }
    }
    std::string Name() const override { return "BubbleSort"; }
};

class QuickSort : public SortStrategy {
public:
    void Sort(std::vector<int>& data) const override {
        QuickSortImpl(data, 0, data.size() - 1);
    }
    std::string Name() const override { return "QuickSort"; }

private:
    void QuickSortImpl(std::vector<int>& data, int low, int high) const {
        if (low >= high) return;
        int pivot = data[high];
        int i = low - 1;
        for (int j = low; j < high; ++j) {
            if (data[j] <= pivot) {
                std::swap(data[++i], data[j]);
            }
        }
    }
};
```

```

    }
    std::swap(data[++i], data[high]);
    QuickSortImpl(data, low, i - 1);
    QuickSortImpl(data, i + 1, high);
}
};

class STLSort : public SortStrategy {
public:
    void Sort(std::vector<int>& data) const override {
        std::sort(data.begin(), data.end());
    }
    std::string Name() const override { return "STLSort"; }
};

// Context
class DataProcessor {
public:
    explicit DataProcessor(std::unique_ptr<SortStrategy> strategy)
        : strategy_(std::move(strategy)) {}

    void SetStrategy(std::unique_ptr<SortStrategy> strategy) {
        strategy_ = std::move(strategy);
    }

    void Process(std::vector<int> data) {
        std::cout << "Using " << strategy_>Name() << std::endl;
        strategy_>Sort(data);
        for (int v : data) std::cout << v << " ";
        std::cout << std::endl;
    }

private:
    std::unique_ptr<SortStrategy> strategy_;
};

void TestDynamicStrategy() {
    std::vector<int> data = {5, 2, 8, 1, 9, 3};

    DataProcessor processor(std::make_unique<BubbleSort>());
    processor.Process(data); // BubbleSort

    processor.SetStrategy(std::make_unique<QuickSort>());
    processor.Process(data); // QuickSort

    processor.SetStrategy(std::make_unique<STLSort>());
    processor.Process(data); // STLSort
}

```

B. 静态策略 (Policy-Based Design, 编译期多态)

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

// =====
// 静态策略 - 模板策略 (Policy-Based Design)
// 对标 std::sort 的自定义 Comparator
// =====

// 策略标签
struct Ascending {
    template <typename T>
    bool operator()(const T& a, const T& b) const {
        return a < b;
    }
    static std::string Name() { return "Ascending"; }
};

struct Descending {
    template <typename T>
    bool operator()(const T& a, const T& b) const {
        return a > b;
    }
    static std::string Name() { return "Descending"; }
};

struct AbsDescending {
    template <typename T>
    bool operator()(const T& a, const T& b) const {
        return std::abs(a) > std::abs(b);
    }
    static std::string Name() { return "AbsDescending"; }
};

// Context - 编译期确定策略, 零开销
template <typename ComparePolicy = Ascending>
class StaticDataProcessor {
public:
    void Process(std::vector<int> data) {
        std::cout << "Using " << ComparePolicy::Name() << std::endl;
        std::sort(data.begin(), data.end(), ComparePolicy{});
        for (int v : data) std::cout << v << " ";
        std::cout << std::endl;
    }
};

void TestStaticStrategy() {
    std::vector<int> data = {5, -2, 8, -1, 9, -3};

    StaticDataProcessor<Ascending> asc;
```

```
asc.Process(data); // -3 -2 -1 5 8 9

StaticDataProcessor<Descending> desc;
desc.Process(data); // 9 8 5 -1 -2 -3

StaticDataProcessor<AbsDescending> absDesc;
absDesc.Process(data); // 9 8 5 -3 -2 -1
}

// 与 std::sort 对比:
// std::sort 通过函数对象作为策略，正是静态策略模式的经典应用
```

5.4 使用场景

- 排序算法选择 (std::sort + comparator)
- 压缩算法选择 (gzip / bzip2 / lz4)
- 支付方式选择 (微信/支付宝/银行卡)
- 身份验证策略 (密码/短信/生物识别)
- 游戏 AI 行为切换 (巡逻/追击/逃跑)
- 缓存淘汰策略 (LRU/LFU/FIFO)

5.5 优缺点

优点:

- 算法可互换，客户端只需切换策略对象
- 避免大量 if-else/switch，符合开闭原则
- 策略可独立测试
- 静态策略（模板）零运行时开销

缺点:

- 客户端必须知道有哪些策略（违背最少知识原则的部分）
- 策略类数量增多
- 动态策略有虚函数开销
- 策略之间共享数据需要额外的上下文传递

5.6 面试题

Q1: 策略模式 vs 模板方法模式？

维度	策略模式	模板方法
复用方式	组合 (has-a)	继承 (is-a)
粒度	替换整个算法	替换算法的某一步骤
耦合	策略与 Context 通过接口解耦	子类依赖父类骨架

维度	策略模式	模板方法
C++实现	虚接口 / 模板	NVI (非虚接口)
运行时切换	可以	不能 (继承关系编译期固定)

Q2: 什么时候用静态策略，什么时候用动态策略？

- **静态策略**：编译期能确定策略，追求零开销 → `std::sort` comparator
- **动态策略**：需要运行时切换策略（如用户设置、配置文件） → 虚函数

C++ 项目常见做法：先用虚函数写出清晰接口，性能分析确认瓶颈后改为模板。

Q3: 策略模式如何避免策略膨胀？

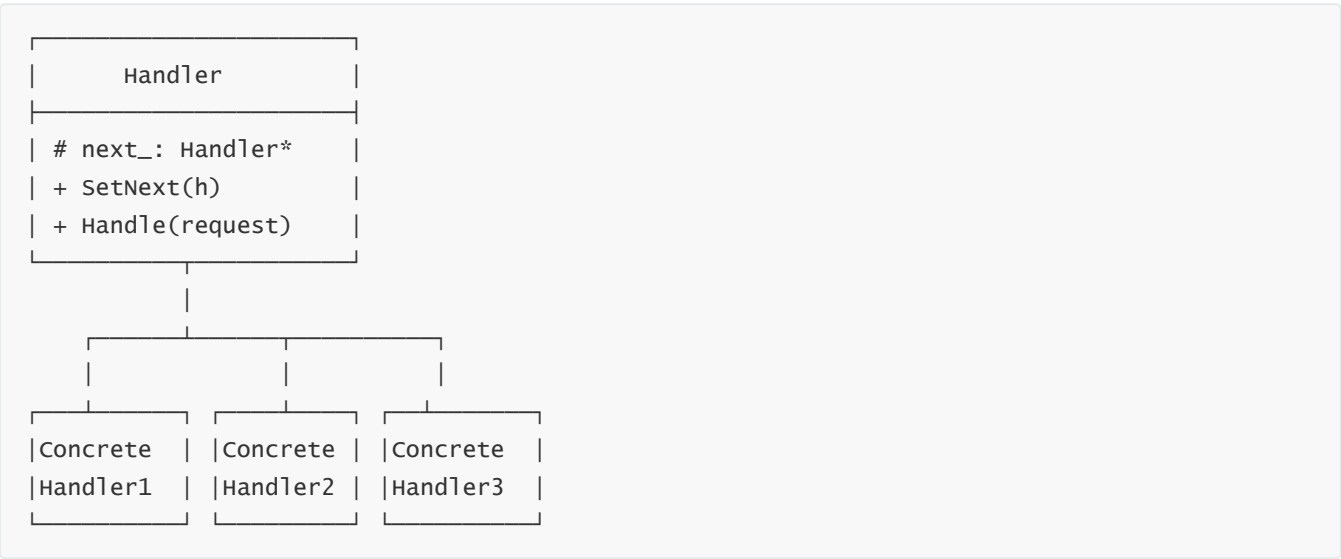
1. 提取"不变部分"到 Context，策略只定义差异
2. 使用函数式组合： `std::function<void(Context&)>` 代替策略接口
3. Lambda 表达式作为轻量级策略（类似 `sort` 的 `comparator`）
4. Builder + 策略：用建造者模式组装参数化的策略对象

6. 责任链模式 (Chain of Responsibility)

6.1 思想

将请求的发送者和接收者解耦，让多个对象都有机会处理请求。 将这些对象连成一条链，请求沿链传递直到被处理。

6.2 类图



6.3 代码实现

A. 链表方式 (传统)

```
#include <iostream>
#include <memory>
#include <string>

// =====
// 责任链 - 审批流程
// =====

// 请求
struct PurchaseRequest {
    double amount;
    std::string purpose;
};

// 抽象处理者
class Approver {
public:
    virtual ~Approver() = default;

    void SetNext(std::shared_ptr<Approver> next) {
        next_ = next;
    }

    virtual void HandleRequest(const PurchaseRequest& request) {
        if (next_) {
            next_>HandleRequest(request);
        } else {
            std::cout << "[CHAIN END] Request for $"
                << request.amount << " not approved." << std::endl;
        }
    }

protected:
    std::shared_ptr<Approver> next_;
};

// 具体处理者: Team Lead (可批 <= $1000)
class TeamLead : public Approver {
public:
    void HandleRequest(const PurchaseRequest& request) override {
        if (request.amount <= 1000) {
            std::cout << "[TeamLead] Approved $" << request.amount
                << " for " << request.purpose << std::endl;
        } else {
            std::cout << "[TeamLead] Cannot approve $" << request.amount
                << ", escalating..." << std::endl;
            Approver::HandleRequest(request); // 传递给下一级
        }
    }
};
```

```

    }
};

// Manager (可批 <= $5000)
class Manager : public Approver {
public:
    void HandleRequest(const PurchaseRequest& request) override {
        if (request.amount <= 5000) {
            std::cout << "[Manager] Approved $" << request.amount
                << " for " << request.purpose << std::endl;
        } else {
            std::cout << "[Manager] Cannot approve $" << request.amount
                << ", escalating..." << std::endl;
            Approver::HandleRequest(request);
        }
    }
};

// Director (可批 <= $20000)
class Director : public Approver {
public:
    void HandleRequest(const PurchaseRequest& request) override {
        if (request.amount <= 20000) {
            std::cout << "[Director] Approved $" << request.amount
                << " for " << request.purpose << std::endl;
        } else {
            std::cout << "[Director] Cannot approve $" << request.amount
                << ", escalating..." << std::endl;
            Approver::HandleRequest(request);
        }
    }
};

// CEO (无上限)
class CEO : public Approver {
public:
    void HandleRequest(const PurchaseRequest& request) override {
        std::cout << "[CEO] Approved $" << request.amount
            << " for " << request.purpose
            << " (unlimited authority)" << std::endl;
    }
};

void TestChainOfResponsibility() {
    auto teamLead = std::make_shared<TeamLead>();
    auto manager = std::make_shared<Manager>();
    auto director = std::make_shared<Director>();
    auto ceo = std::make_shared<CEO>();

    // 构建责任链
    teamLead->SetNext(manager);
    manager->SetNext(director);
    director->SetNext(ceo);
}

```

```

std::cout << "=== Request A: $500 ===" << std::endl;
teamLead->HandleRequest({500, "office supplies"});

std::cout << "\n=== Request B: $4000 ===" << std::endl;
teamLead->HandleRequest({4000, "Team building event"});

std::cout << "\n=== Request C: $15000 ===" << std::endl;
teamLead->HandleRequest({15000, "New server"});

std::cout << "\n=== Request D: $50000 ===" << std::endl;
teamLead->HandleRequest({50000, "Department relocation"});
}

```

B. 组合方式 (变体)

```

// 组合式责任链：无需手动链接，按注册顺序自动传递
class CompositeProcessor {
public:
    void AddHandler(std::unique_ptr<Approver> handler) {
        if (!handlers_.empty()) {
            handlers_.back()->SetNext(handler.get()); // 注意生命周期
        }
        handlers_.push_back(std::move(handler));
    }

    void Process(const PurchaseRequest& request) {
        if (!handlers_.empty()) {
            handlers_.front()->HandleRequest(request);
        }
    }

private:
    std::vector<std::unique_ptr<Approver>> handlers_; // 持有所有权
};

```

6.4 使用场景

- 审批流程：逐级审批 (TL → Manager → Director → CEO)
- 中间件/拦截器：Web 框架的 middleware pipeline
- 异常处理链：try-catch 的异常匹配
- 日志系统：日志级别过滤器链
- 事件处理：UI 事件冒泡
- 协议栈：OSI 七层网络模型中的层层封装与解析

6.5 优缺点

优点：

- 解耦：请求发送者不需要知道哪个接收者处理
- 灵活：动态调整链的结构
- 单一职责：每个 Handler 只处理自己能处理的部分

缺点：

- 请求可能未被处理（链尾无默认处理器）
- 链太长导致性能问题（遍历开销）
- 调试困难：不知道请求在哪一环被处理
- 循环引用风险（如果链形成环）

6.6 面试题

Q1: 责任链和装饰器模式有什么区别？

维度	责任链	装饰器
目的	请求传递，任一处理	动态增强功能
处理	某一节点处理或传递	每层都执行
中断	可以中断传递	通常不中断
责任	独立处理或转发	增强后用 super 传递

Q2: 责任链如何处理请求无匹配的情况？

1. 在链尾加默认处理器（Null Object 模式）
2. 返回值表示处理状态，发送者检查
3. 抛出异常
4. 使用日志告警

```
class DefaultHandler : public Approver {
public:
    void HandleRequest(const PurchaseRequest& request) override {
        std::cerr << "WARNING: No handler for $" << request.amount << std::endl;
    }
};
```

Q3: 链中某个处理器如何修改请求？

可以将 `HandleRequest` 签名改为 `PurchaseRequest&`（非 `const`），允许中间节点修改请求内容，如添加审批意见、扣减预算等。

但要注意：修改请求可能导致后续处理行为变化，需要文档明确说明。

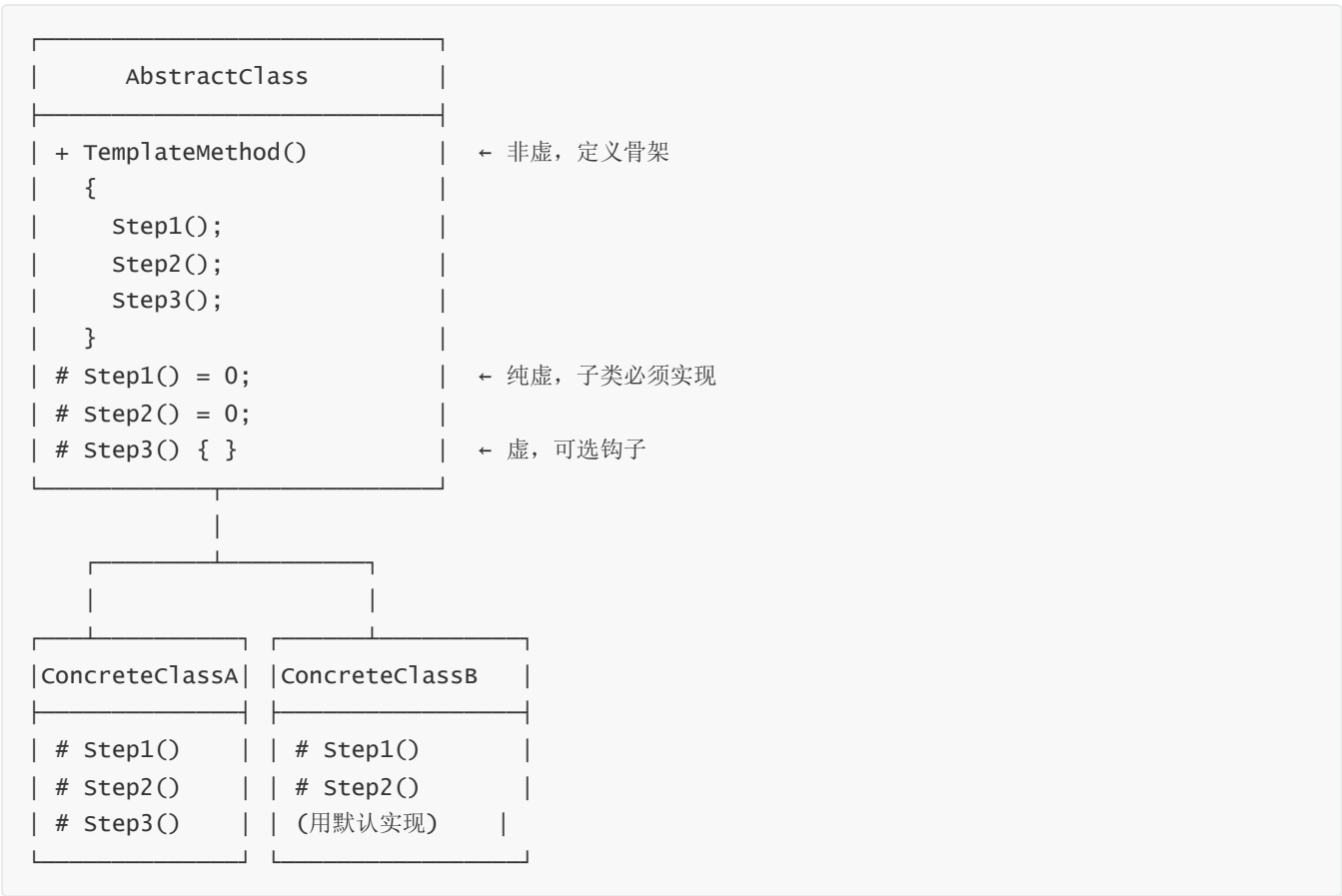
7. 模板方法模式 (Template Method)

7.1 思想

定义一个操作中的算法骨架，将某些步骤延迟到子类实现。模板方法让子类在不改变算法结构的情况下，重新定义某些步骤。

核心：NVI (Non-Virtual Interface) 惯用法 — 公开的非虚接口调用私有的虚函数。

7.2 类图



7.3 代码实现

A. 基础模板方法

```
#include <iostream>
#include <string>
#include <chrono>
#include <thread>

// =====
// 模板方法 - 数据导入框架
// =====

class DataImporter {
public:
    virtual ~DataImporter() = default;
```

```

// 模板方法 - 定义算法骨架 (NVI 惯用法)
void Import(const std::string& filePath) {
    std::cout << "[Import] Starting import from: " << filePath << std::endl;

    if (!ValidateFile(filePath)) {
        std::cout << "[Import] File validation failed!" << std::endl;
        return;
    }

    auto start = std::chrono::steady_clock::now();

    auto rawData = ReadData(filePath);          // 子类实现
    auto parsedData = ParseData(rawData);       // 子类实现
    SaveToDatabase(parsedData);                 // 子类实现

    auto end = std::chrono::steady_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
    std::cout << "[Import] Completed in " << duration.count() << "ms" << std::endl;

    // 钩子: 子类可选覆盖的后处理
    PostProcess();
}

protected:
// 原始操作 - 子类必须实现
virtual std::string ReadData(const std::string& filePath) = 0;
virtual std::string ParseData(const std::string& rawData) = 0;
virtual void SaveToDatabase(const std::string& parsedData) = 0;

// 钩子操作 - 子类可选覆盖
virtual bool validateFile(const std::string& filePath) {
    // 默认实现: 检查文件是否为空
    return !filePath.empty();
}

virtual void PostProcess() {
    // 默认什么都不做
}
};

// ----- CSV 导入器 -----
class CSVImporter : public DataImporter {
protected:
    std::string ReadData(const std::string& filePath) override {
        std::cout << "[CSV] Reading from " << filePath << std::endl;
        return "name,age\nAlice,30\nBob,25"; // 模拟读取
    }

    std::string ParseData(const std::string& rawData) override {
        std::cout << "[CSV] Parsing CSV data" << std::endl;
        // 模拟解析
        return "PARSED_CSV: " + rawData;
    }
}

```

```

    void SaveToDatabase(const std::string& parsedData) override {
        std::cout << "[CSV] Saving to DB: " << parsedData << std::endl;
    }
};

// ----- JSON 导入器 -----
class JSONImporter : public DataImporter {
protected:
    std::string ReadData(const std::string& filePath) override {
        std::cout << "[JSON] Reading from " << filePath << std::endl;
        return R"({"users":[{"name":"Alice","age":30}]})";
    }

    std::string ParseData(const std::string& rawData) override {
        std::cout << "[JSON] Parsing JSON data" << std::endl;
        return "PARSED_JSON: " + rawData;
    }

    void SaveToDatabase(const std::string& parsedData) override {
        std::cout << "[JSON] Saving to DB" << std::endl;
    }

    // 覆盖钩子
    void PostProcess() override {
        std::cout << "[JSON] Running JSON-specific post-processing" << std::endl;
    }
};

void TestTemplateMethod() {
    std::cout << "=== CSV Import ===" << std::endl;
    CSVImporter csv;
    csv.Import("data.csv");

    std::cout << "\n=== JSON Import ===" << std::endl;
    JSONImporter json;
    json.Import("data.json");
}

```

B. NVI (Non-Virtual Interface) 惯用法详解

```

// =====
// NVI 惯用法 - C++ 模板方法的标准实现
// =====

class Game {
public:
    // NVI: 公开接口是非虚的
    void Run() {
        Initialize();
        while (!IsGameOver()) {
            Update();
        }
    }
}

```

```

        Render();
    }
    Cleanup();
}

virtual ~Game() = default;

private: // 注意：虚函数是 private 的!
    virtual void Initialize() { std::cout << "Game::Initialize\n"; }
    virtual bool IsGameOver() { return false; }
    virtual void Update() = 0; // 纯虚，强制子类实现
    virtual void Render() = 0;
    virtual void Cleanup() { std::cout << "Game::Cleanup\n"; }
};

class Chess : public Game {
private: // 子类也不用 public
    void Initialize() override {
        Game::Initialize(); // 可调用基类实现
        std::cout << "Chess::Initialize - set up board\n";
    }
    bool IsGameOver() override { return moveCount_ > 100; }
    void Update() override { std::cout << "Chess::Update\n"; ++moveCount_; }
    void Render() override { std::cout << "Chess::Render board\n"; }
    void Cleanup() override { std::cout << "Chess::Cleanup - save replay\n"; }
    int moveCount_ = 0;
};

// 使用 NVI 的好处：
// 1. 基类控制调用顺序，保证预处理/后处理不被子类绕过
// 2. 基类可以在调用虚函数前后加锁、打日志、计时
// 3. 基类可以先做参数校验再调用虚函数
// 4. 接口和实现的关注点分离

```

7.4 使用场景

- 框架设计：Spring、Qt、MFC 大量使用模板方法
- 数据导入/导出流程
- 测试框架：setUp → testMethod → tearDown
- 构建流程：初始化 → 编译 → 链接 → 打包
- 游戏循环：Init → Input → Update → Render → Cleanup
- 算法框架：排序、搜索等算法的骨架

7.5 优缺点

优点：

- 代码复用：骨架代码只在基类中出现一次
- 控制反转（好莱坞原则）：基类调用子类，而非子类调用基类
- 扩展性强：新增子类只需实现差异部分

- NVI 增强：基类可做前置/后置处理（加锁、计时、日志）

缺点：

- 继承的脆弱性：基类修改骨架可能破坏所有子类
- 灵活性受限：算法结构固定，不能运行时改变
- 深层次继承导致代码难以理解
- 子类必须实现所有纯虚函数，即使不需要

7.6 面试题

Q1: 模板方法模式 vs 策略模式？

这是面试极高频对比题。

维度	模板方法	策略模式
复用机制	继承	组合
控制	基类控制骨架，调用子类	Context 调用策略算法
粒度	替换算法的个别步骤	替换整个算法
编译时	固定	可运行时切换
C++ 实现	NVI (非虚接口)	虚接口/模板
原则	好莱坞原则	开闭原则 + 组合优于继承
何时用	流程固定，步骤不同	算法可变且需独立

Q2: 为什么 NVI 中虚函数通常声明为 private？

1. 强制用户通过公共非虚接口调用（保证前置/后置处理不被绕过）
2. 子类不需要公开这些虚函数
3. 接口与实现分离：public 定义"做什么", private virtual 定义"怎么做"
4. 减少子类对外暴露的接口面

Q3: 模板方法中"钩子函数"的作用？

钩子（Hook）是基类中提供默认空实现的虚函数，子类可选覆盖：

```
// 钩子：基类提供默认行为，子类可以"挂载"自定义行为
virtual void OnBeforeSave() {} // 保存前的校验钩子
virtual void OnProgress(int pct) {} // 进度报告钩子
```

用途：在不修改骨架的前提下，允许子类在固定节点插入自定义逻辑。与纯虚函数不同，钩子不强制子类实现。

8. 其他模式简述

8.1 适配器模式 (Adapter)

思想： 将一个接口转换成客户期望的另一个接口，使原本不兼容的类可以一起工作。

```
// 目标接口
class ITarget {
public:
    virtual void Request() = 0;
    virtual ~ITarget() = default;
};

// 被适配者（不兼容的旧接口）
class Adaptee {
public:
    void SpecificRequest() {
        std::cout << "Adaptee::SpecificRequest" << std::endl;
    }
};

// 对象适配器（使用组合）
class Adapter : public ITarget {
public:
    explicit Adapter(std::shared_ptr<Adaptee> adaptee) : adaptee_(adaptee) {}
    void Request() override {
        adaptee_->SpecificRequest(); // 委托调用
    }
private:
    std::shared_ptr<Adaptee> adaptee_;
};

// 使用: std::unique_ptr<ITarget> target = std::make_unique<Adapter>(adaptee);
```

C++ 中特别注意： STL 的 `std::stack`、`std::queue` 就是适配器模式（底层容器默认为 `std::deque`）。

8.2 装饰器模式 (Decorator)

思想： 动态地给对象添加额外职责，比继承更灵活。

```
// 组件接口
class Coffee {
public:
    virtual double Cost() const = 0;
    virtual std::string Description() const = 0;
    virtual ~Coffee() = default;
};

// 具体组件
class SimpleCoffee : public Coffee {
public:
    double Cost() const override { return 5.0; }
    std::string Description() const override { return "Simple Coffee"; }
```

```
};

// 装饰器基类
class CoffeeDecorator : public Coffee {
public:
    explicit CoffeeDecorator(std::unique_ptr<Coffee> coffee)
        : coffee_(std::move(coffee)) {}
protected:
    std::unique_ptr<Coffee> coffee_;
};

// 具体装饰器
class MilkDecorator : public CoffeeDecorator {
public:
    using CoffeeDecorator::CoffeeDecorator;
    double Cost() const override { return coffee_>Cost() + 1.5; }
    std::string Description() const override {
        return coffee_>Description() + ", Milk";
    }
};

class SugarDecorator : public CoffeeDecorator {
public:
    using CoffeeDecorator::CoffeeDecorator;
    double Cost() const override { return coffee_>Cost() + 0.5; }
    std::string Description() const override {
        return coffee_>Description() + ", Sugar";
    }
};

// 使用：嵌套装饰
// auto coffee = std::make_unique<MilkDecorator>(
//     std::make_unique<SugarDecorator>(
//         std::make_unique<SimpleCoffee>()
//     )
// );
// coffee->Cost() == 7.0
```

与代理模式的区别：装饰器增强功能，代理控制访问。

8.3 代理模式 (Proxy)

思想：为另一个对象提供替身或占位符来控制访问。

常见类型：

- **远程代理：**为远程对象提供本地代表（RPC stub）
- **虚拟代理：**延迟加载大对象（图片占位符）
- **保护代理：**控制权限访问
- **智能指针：** `std::shared_ptr` 就是代理模式的应用

```
// 虚拟代理：延迟加载图片
```

```

class IImage {
public:
    virtual void Display() = 0;
    virtual ~IImage() = default;
};

class RealImage : public IImage {
public:
    RealImage(const std::string& path) : path_(path) {
        std::cout << "Loading image from " << path_ << " (expensive!)" << std::endl;
    }
    void Display() override {
        std::cout << "Displaying " << path_ << std::endl;
    }
private:
    std::string path_;
};

class ImageProxy : public IImage {
public:
    ImageProxy(const std::string& path) : path_(path) {}
    void Display() override {
        if (!realImage_) {
            realImage_ = std::make_unique<RealImage>(path_); // 延迟加载
        }
        realImage_>Display();
    }
private:
    std::string path_;
    std::unique_ptr<RealImage> realImage_;
};

```

C++ 中特别注意:

- `std::shared_ptr` — 引用计数代理
- `std::weak_ptr` — 观察者代理
- Pimpl (Pointer to Implementation) — 编译防火墙代理

8.4 建造者模式 (Builder)

思想: 将复杂对象的构建与表示分离，同样的构建过程可以创建不同的表示。

```

// 产品
class Computer {
public:
    void SetCPU(const std::string& cpu) { cpu_ = cpu; }
    void SetRAM(const std::string& ram) { ram_ = ram; }
    void SetStorage(const std::string& s) { storage_ = s; }
    void SetGPU(const std::string& gpu) { gpu_ = gpu; }

    void Show() const {
        std::cout << "Computer: " << cpu_ << " / " << ram_

```

```

        << " / " << storage_ << " / " << gpu_ << std::endl;
    }
private:
    std::string cpu_ = "none";
    std::string ram_ = "none";
    std::string storage_ = "none";
    std::string gpu_ = "none";
};

// 抽象建造者
class ComputerBuilder {
public:
    virtual ~ComputerBuilder() = default;
    virtual void BuildCPU() = 0;
    virtual void BuildRAM() = 0;
    virtual void BuildStorage() = 0;
    virtual void BuildGPU() = 0;
    Computer& GetResult() { return computer_; }
protected:
    Computer computer_;
};

// 具体建造者: 游戏电脑
class GamingComputerBuilder : public ComputerBuilder {
public:
    void BuildCPU() override { computer_.SetCPU("Intel i9-13900K"); }
    void BuildRAM() override { computer_.SetRAM("64GB DDR5"); }
    void BuildStorage() override { computer_.SetStorage("2TB NVMe SSD"); }
    void BuildGPU() override { computer_.SetGPU("NVIDIA RTX 4090"); }
};

// 具体建造者: 办公电脑
class OfficeComputerBuilder : public ComputerBuilder {
public:
    void BuildCPU() override { computer_.SetCPU("Intel i5-13400"); }
    void BuildRAM() override { computer_.SetRAM("16GB DDR4"); }
    void BuildStorage() override { computer_.SetStorage("512GB SSD"); }
    void BuildGPU() override { computer_.SetGPU("Integrated Graphics"); }
};

// Director
class Director {
public:
    void Construct(ComputerBuilder& builder) {
        builder.BuildCPU();
        builder.BuildRAM();
        builder.BuildStorage();
        builder.BuildGPU();
    }
};

// 使用:
// Director director;

```

```
// GamingComputerBuilder gamingBuilder;  
// director.Construct(gamingBuilder);  
// gamingBuilder.GetResult().Show();
```

C++ 中特别注意：Builder 模式常和链式调用（Fluent Interface）结合，允许 `builder.SetCPU().SetRAM().Build()` 风格。

8.5 享元模式 (Flyweight) — `std::string` SSO

思想：运用共享技术有效支持大量细粒度对象，通过分离内部状态（Intrinsic）和外部状态（Extrinsic）来减少内存。

```
#include <unordered_map>  
#include <string>  
  
// 享元工厂  
class GlyphFactory {  
public:  
    // 共享字符对象（内部状态：字符编码）  
    // 外部状态（位置、字体、颜色）由客户端维护  
    static char GetCharacter(char c) {  
        // 内部状态不变，多次调用返回同一对象  
        return c; // 简化：实际可能是共享的 Glyph 对象  
    }  
};  
  
// C++ 核心案例：std::string 的 SSO (Small String Optimization)  
//  
// 大多数 std::string 实现对于短字符串（如 <= 15 字符）  
// 会把数据存储在 string 对象内部的栈缓冲区中，不分配堆内存。  
// 这本质上是一种享元模式：短串共享 string 对象本身的内存，  
// 避免了大量小对象分配堆的开销。  
//  
// 面试金句：  
// "std::string 的 SSO 是享元模式的体现 -  
// 短字符串共享对象内部的缓冲区（内部状态），避免堆分配。"
```

使用场景：

- 文字编辑器中的字符对象（每个字符不独立分配内存）
 - 游戏中的粒子效果（共享纹理、几何信息）
 - 数据库连接池中的连接对象
 - `std::string` SSO
 - `std::shared_ptr` 的控制块共享
-

9. 设计模式对比总结

9.1 核心模式对比表

模式	类型	意图	何时使用	C++ 特别考量
Singleton	创建型	确保唯一实例	全局配置、日志、连接池	Meyers Singleton (C++11 static local); 注意析构顺序
Factory Method	创建型	子类决定创建哪个对象	框架扩展点, 运行时选类型	可用模板替代虚函数 (编译期多态)
Abstract Factory	创建型	创建产品族	跨平台UI、多数数据库后端	结合模板减少虚函数开销; 注意对象生命周期
Builder	创建型	分离构建与表示	复杂对象分步创建	Fluent API; 移动语义优化
Observer	行为型	一对多状态通知	事件系统、MVC、消息推送	<code>weak_ptr</code> 防悬空; 信号槽替代; 重入保护
Strategy	行为型	算法可互换	排序、压缩、支付策略	静态策略 (模板零开销) vs 动态策略 (虚函数)
Chain of Resp.	行为型	请求沿链传递	审批流、中间件管道	<code>unique_ptr</code> 管理链的所有权
Template Method	行为型	算法骨架固定	框架设计、流程控制	NVI 惯用法; private virtual
Adapter	结构型	接口转换	兼容遗留代码、第三方库	<code>std::stack / std::queue</code> 就是适配器
Decorator	结构型	动态增强功能	IO流、UI增强	<code>std::iostream</code> 体系就是装饰器
Proxy	结构型	控制访问	延迟加载、权限控制、RPC	Pimpl; <code>shared_ptr / weak_ptr</code> ; 移动语义
Flyweight	结构型	共享细粒度对象	字符系统、连接池	<code>std::string</code> SSO; <code>shared_ptr</code> 控制块

9.2 易混淆模式对比

对比	区分要点
Factory Method vs Abstract Factory	Factory Method 创建 一种 产品; Abstract Factory 创建 一族 产品
Decorator vs Proxy	Decorator 增强 功能; Proxy 控制 访问
Adapter vs Decorator	Adapter 改变 接口 ; Decorator 不改接口, 增强 行为

对比	区分要点
Strategy vs Template Method	Strategy 用 组合 替换整个算法; Template Method 用 继承 替换步骤
Chain of Resp vs Decorator	Chain 可能 中断 传递; Decorator 每层都执行
Builder vs Factory	Builder 分步 构建复杂对象; Factory 一步 创建

9.3 设计原则速查

设计模式是 SOLID 原则的具体体现。

原则	体现模式
单一职责 (SRP)	Observer 分离通知者和观察者
开闭原则 (OCP)	Strategy、Factory Method、Decorator
里氏替换 (LSP)	Template Method (子类替换基类位置)
接口隔离 (ISP)	Observer 的窄接口
依赖倒置 (DIP)	Abstract Factory (依赖抽象工厂, 不依赖具体工厂)
组合优于继承	Strategy (组合策略对象)、Decorator
最少知识 (LoD)	Facade (外观模式, 未展开)

10. 面试核心

面试重点

1. **能手写** Singleton (Meyers 版本 + DCL 版本)、Factory Method、Observer、Strategy
2. **能画出** 每个模式的类图 (UML 箭头关系)
3. **能对比** 易混淆模式 (Strategy vs Template Method、Decorator vs Proxy、Factory Method vs Abstract Factory)
4. **能举出** C++ 标准库中的应用 (`std::sort` = Strategy、`std::stack` = Adapter、`std::iostream` = Decorator、`std::string` SSO = Flyweight)
5. **能结合** SOLID 原则解释设计模式的价值

高频考点

- Singleton 的线程安全演进 (C++11 前 vs 后)
- NVI 惯用法 (模板方法的 C++ 实现)
- 静态策略 (模板) vs 动态策略 (虚函数)
- 观察者模式的悬空引用问题及解决方案
- 工厂模式在 C++ 中何时用虚函数、何时用模板
- 设计模式的反面: 过度设计 (Over-engineering)

常见陷阱

1. **滥用 Singleton**: 把 Singleton 当全局变量用, 导致测试困难、耦合严重
2. **忽略线程安全**: 在多线程环境使用非线程安全的懒汉 Singleton
3. **忘记虚析构函数**: 工厂模式返回基类指针必须声明虚析构
4. **观察者未注销**: Observer 析构时未 Detach, 导致悬空指针
5. **过度使用继承**: 能用组合 (Strategy) 就不用继承 (Template Method)
6. **过早抽象**: 只有两个变体时就引入工厂模式, 导致过度设计
7. **循环依赖**: 责任链中设置 next 形成环

建议背诵内容

1. Singleton 核心三行:

```
static Singleton& GetInstance() {  
    static Singleton instance;  
    return instance;  
}  
Singleton(const Singleton&) = delete;  
Singleton& operator=(const Singleton&) = delete;
```

2. NVI 核心骨架:

```
class Base {  
public:  
    void Algorithm() { // 非虚、公开  
        Step1(); Step2(); Step3();  
    }  
private:  
    virtual void Step1() = 0; // 虚、私有  
    virtual void Step2() = 0;  
    virtual void Step3() {}  
};
```

3. 策略 vs 模板方法 核心区别:

- 策略 = 组合 + 算法可互换 + 运行时切换
- 模板方法 = 继承 + 骨架固定 + 编译期固定

4. C++ 设计模式金句:

- "用 `= delete` 而不是私有构造函数来禁止拷贝"
- "用 `std::unique_ptr` 而不是裸指针持有工厂创建的对象"
- "用 `std::weak_ptr` 解决观察者悬空引用"
- "静态多态 (模板) 零开销, 动态多态 (虚函数) 运行时灵活"

面试建议: 不要死记硬背代码。理解每个模式解决的**问题**和**变化维度**, 面试时从场景出发推导模式选择, 比背诵定义更能体现工程思维。